

Week 1: Principles of Reproducible & Replicable Data Science

Steve Rozen and Chuan Hong

2026-08-25

The Steps in Research



Population



Question



Hypothesis



Experimental Design



Experimentor



Data



Analysis Plan



Analyst



Code



Estimate



Claim

Patil, Peng and Leek biorXiv 2016

Important concepts of scientific studies

- Publication: A public claim based on a scientific study, usually in the form of a scientific paper.
- Reproducible: Given a population, hypothesis, experimental + analysis design, data, and code, you get the same parameter estimates in a new analysis. Usually refers to being able to access the data and reproduce the analysis
- Replicable Study: Given a new sample from a population, hypothesis, experimental + analysis design, you get consistent estimates when you redo the analysis. The definition of the population is a key issue
- False Discovery: The claim at the conclusion of a scientific study is different from the claim you would make if you could observe all data from the population given your hypothesis, and experimental + analysis design.
- P-hacking: Given a population, hypothesis, experimental + analysis design, the analysis changes the analysis to match a desired conclusion. Can involve selective omission of data sets, verges on outright fraud
- Fraud and fabricated results

Reproducibility vs Replication *Reproducibility*



Population



Question



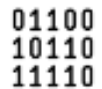
Hypothesis



Experimental Design



Experimentor



Data



Analysis Plan



Analyst



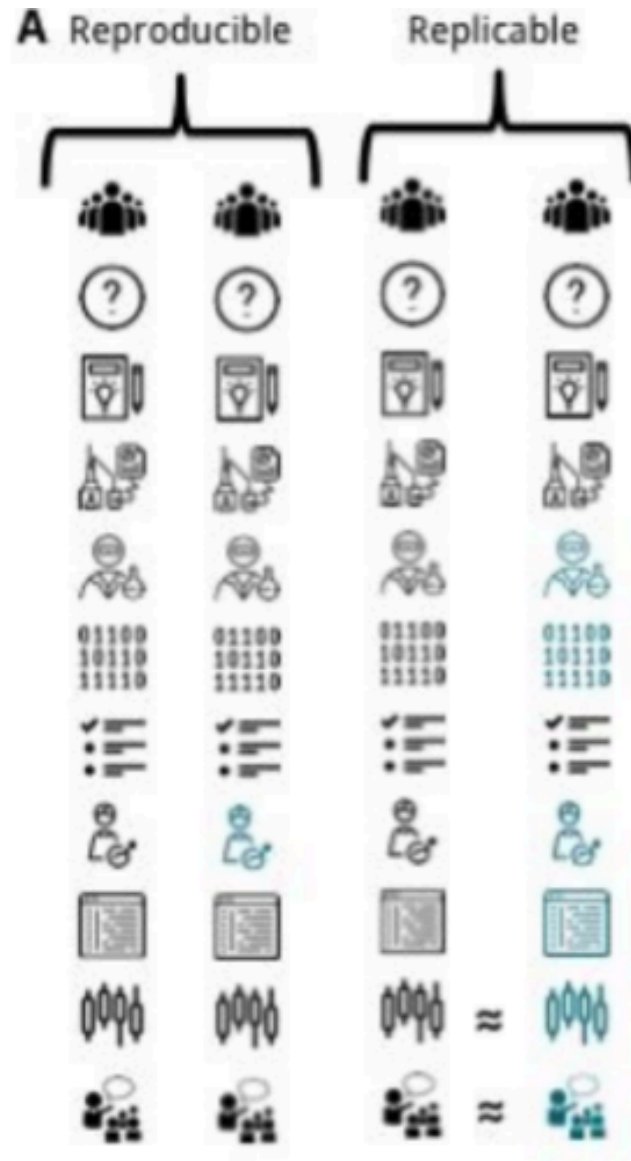
Code



Estimate



Claim



Article | Published: 22 October 2006

Genomic signatures to guide the use of chemotherapeutics

[Anil Potti](#), [Holly K Dressman](#), [Andrea Bild](#), [Richard F Riedel](#), [Gina Chan](#), [Robyn Sayer](#), [Janiel Cragun](#), [Hope Cottrill](#), [Michael J Kelley](#), [Rebecca Petersen](#), [David Harpole](#), [Jeffrey Marks](#), [Andrew Berchuck](#), [Geoffrey S Ginsburg](#), [Phillip Febbo](#), [Johnathan Lancaster](#) & [Joseph R Nevins](#) 

[Nature Medicine](#) **12**, 1294–1300 (2006) | [Cite this article](#)

7616 Accesses | **437** Citations | **98** Altmetric | [Metrics](#)

 A [Retraction](#) to this article was published on 07 January 2011

 A [Corrigendum](#) to this article was published on 01 August 2008

 A [Corrigendum](#) to this article was published on 01 November 2007

 A [Correspondence](#) to this article was published on 01 November 2007

 This article has been [updated](#)

Obstacles to reproducibility

- Data not well organized
- Statistical analysis plan (SAP) not clearly documented
- Code not executable or no code (point-and-click analysis without logging) or based on no-longer-available software libraries
- Figures done by hand (e.g. edited in Photoshop, Adobe Illustrator)
- Data governance restrictions on access to the data
- Hard-to-diff formats (MS word, Excel, PDFs, jupyter notebooks with embedded output)
- Making analysis reproducible takes substantial extra effort on data and code organization

Reproducibility Do's and Don'ts

Do's	Don'ts
Identify Source Data	Only Save Output
Automate Processes	Do Things by Hand
Document Code	Point & Click
Use Version Control	
Track Software Environment	
Set Random Number Generator	
Consider Full Pipeline	

Strategies for Reproducibility

- Data Provenance
- Code Documentation
- Executable Reports
- Version Control

Code Documentation

```
1 #####
2 ### 8/28/2024 This is a sample program to illustrate documentation #
3 ### Assess convergence of empirical mean to theoretical mean      #
4 ### for different distributions                                    #
5 ### Distributions Applied: Normal, Poisson, Exponential          #
6 ### Tuning Parameter: vary sample size 'n'                      #
7 #####
8
9 n <- 100
10
11 rnorm(n,0,1) ### should be \mu
12 rpois(n, 2)  ### should be \lambda
13 rexp(n,2)   ### should be \lambda
```

Setting Random Seeds in R

- `set.seed(1234)` fixes the single stream used by the main R process
- That is enough for code that runs sequentially in one process
- It is **not** enough once work is split across worker processes (e.g. `foreach %dofuture%`), because:
 - The number of workers is often decided at run time (available cores), not fixed in the code
 - Each worker would otherwise draw from the same stream, or from streams seeded by process start time, producing results that change between runs and between machines
- Fix: use an RNG (random number generator) designed for parallel streams, and let the framework assign one independent stream per task, however many tasks or workers there turn out to be

```
1 library(doFuture)
2 plan(multisession)           # number of workers decided at run time
3
4 set.seed(1234)               # seeds the parallel-safe stream generator
5
6 results <- foreach(i = 1:20,
7   .options.future = list(seed = TRUE)) %dofuture% {
8   # each task gets its own L'Ecuyer-CMRG stream
9   rnorm(1) # Do something, for example a single random draw
10 }
```

- `.options.future = list(seed = TRUE)` gives every task its own reproducible stream (L'Ecuyer-CMRG), regardless of how many workers `plan()` ends up using
- Re-running with the same `set.seed()` and the same number of tasks reproduces the same `results`, even if the worker count differs between runs

Setting Random Seeds in Python

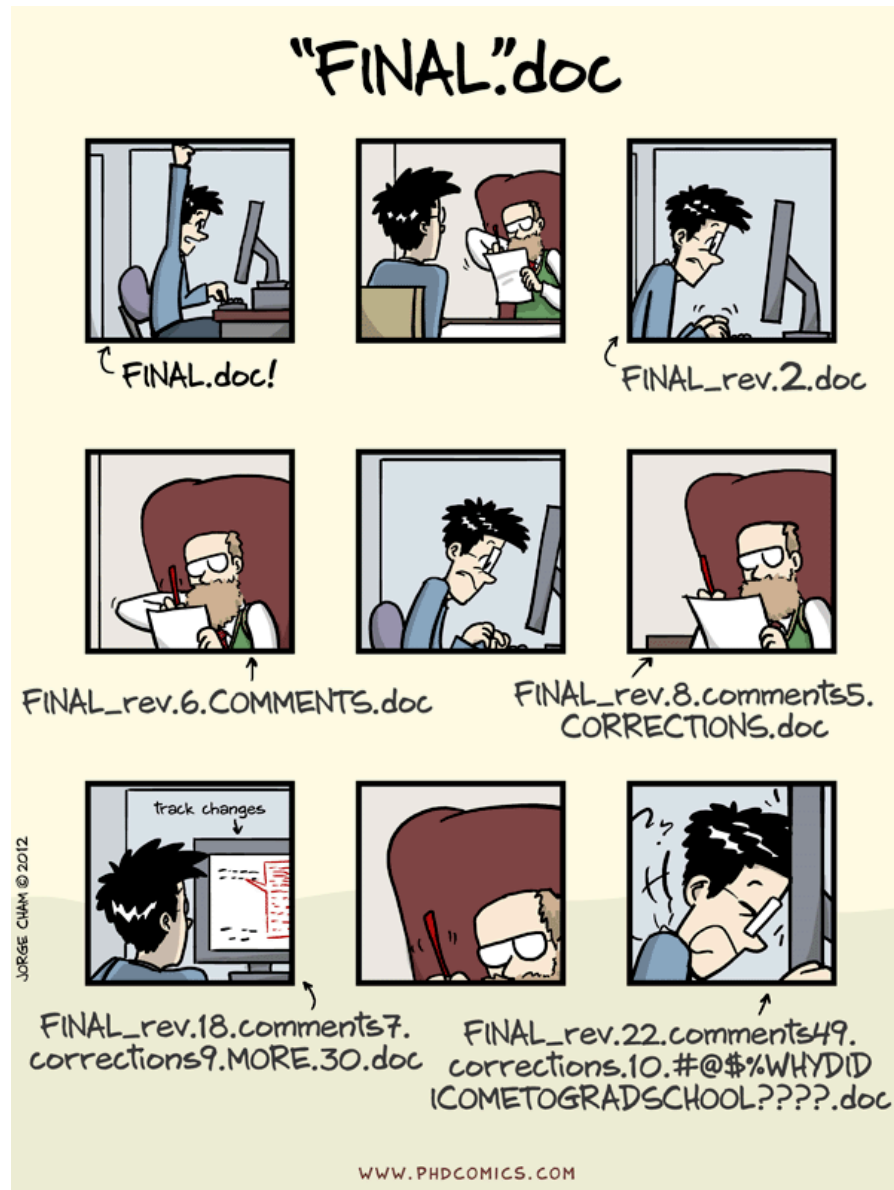
- `numpy.random.seed(1234)` or `random.seed(1234)` fixes a single global stream, same limitation as R's `set.seed()` in a single process
- `multiprocessing.Pool` workers are forked or spawned by the OS; the number of worker processes is a run time choice (e.g. `os.cpu_count()`), not known when the code is written
- Fix: create one `SeedSequence`, then `spawn()` as many independent child seeds as needed, on demand, one per task

```
1 import numpy as np
2 from multiprocessing import Pool
3
4 def draw(seed):
5     rng = np.random.default_rng(seed) # independent stream for this task
6     return rng.normal() # Do something, for example a single random draw
7
8 base = np.random.SeedSequence(1234)
9 child_seeds = base.spawn(20)          # 20 tasks
10
11 with Pool() as pool:                  # pool size (number of workers) decided at run time
12     results = pool.map(draw, child_seeds)
```

Executable reports - Jupyter Notebook and Quarto

- Interactive Computing
 - Write and execute code in real-time within the document
- Multi-Language Support
 - Compatible with languages like Python, R, and Julia
- Better Documentation
 - Combines narrative (rationale, documentation), code, and outputs in one document
- Easy to Share
 - Can be exported to HTML, PDF, and more
- Facilitate Collaboration
 - Ideal with interactive elements and live code

Version Control



Why Use Version Control

- History of changes
- Able to go back to earlier versions (e.g. before you broke something mysterious)
- Merging changes from multiple people
- Run and test on “any” computer

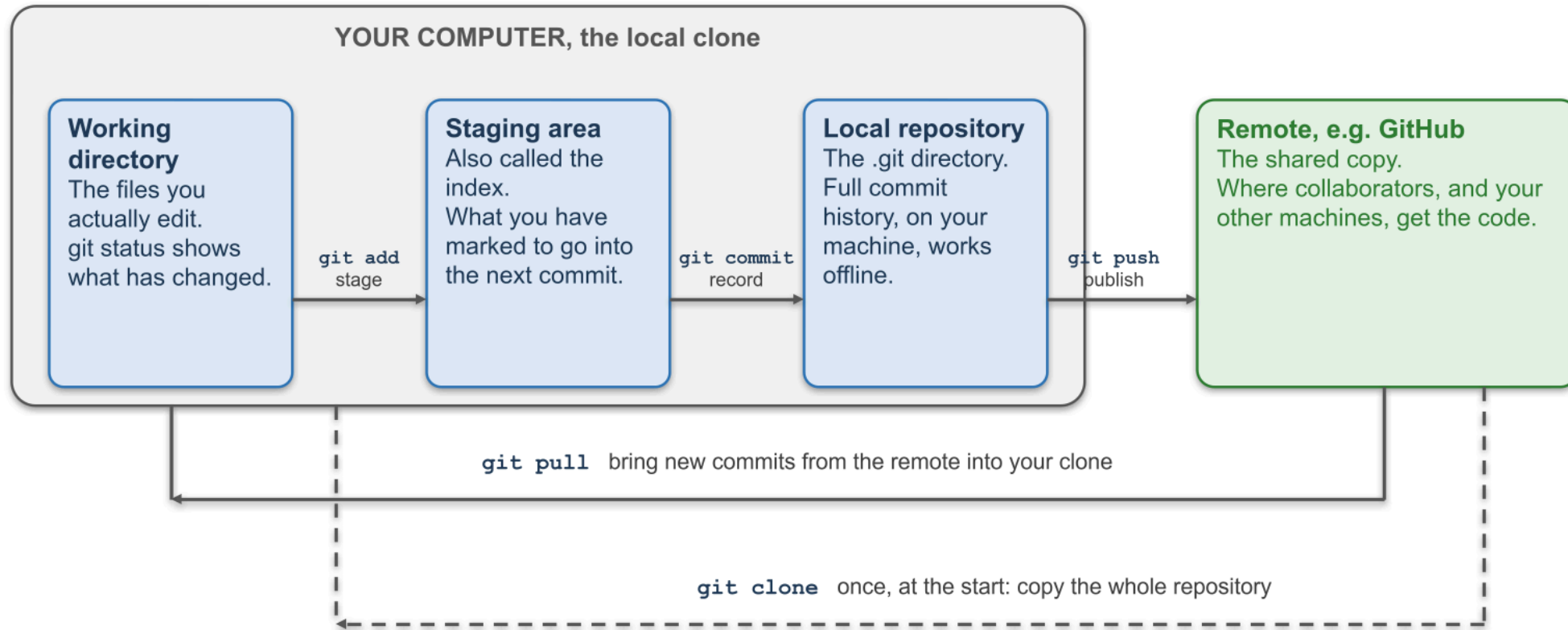
Adapted from Karl Broman lecture notes

Version Control Resources

- GitHub
- Gitlab
- BitBucket

Git

Git: your local clone and the remote



Commits are local until you push. Pull before you start work, so your clone is current because other people are pushing commits to the Remote

HEAD vs the Remote

- **HEAD**: a pointer to your current local commit, usually the tip of the branch you have checked out
- Remote (e.g. **origin**): a reference to another repository, such as one on GitHub
- **origin/main**: your local clone's record of where the remote's **main** branch was as of the last fetch or push
- These diverge in normal use:
 - Commits you make locally move **HEAD** forward, but **origin/main** does not update until you **git push**
 - Commits others push move **origin/main** forward once you **git fetch**, but your **HEAD** does not move until you **git pull** or **git merge**

Reproducibility vs Replication *Replication*



Population



Question



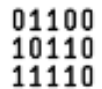
Hypothesis



Experimental Design



Experimentor



Data



Analysis Plan



Analyst



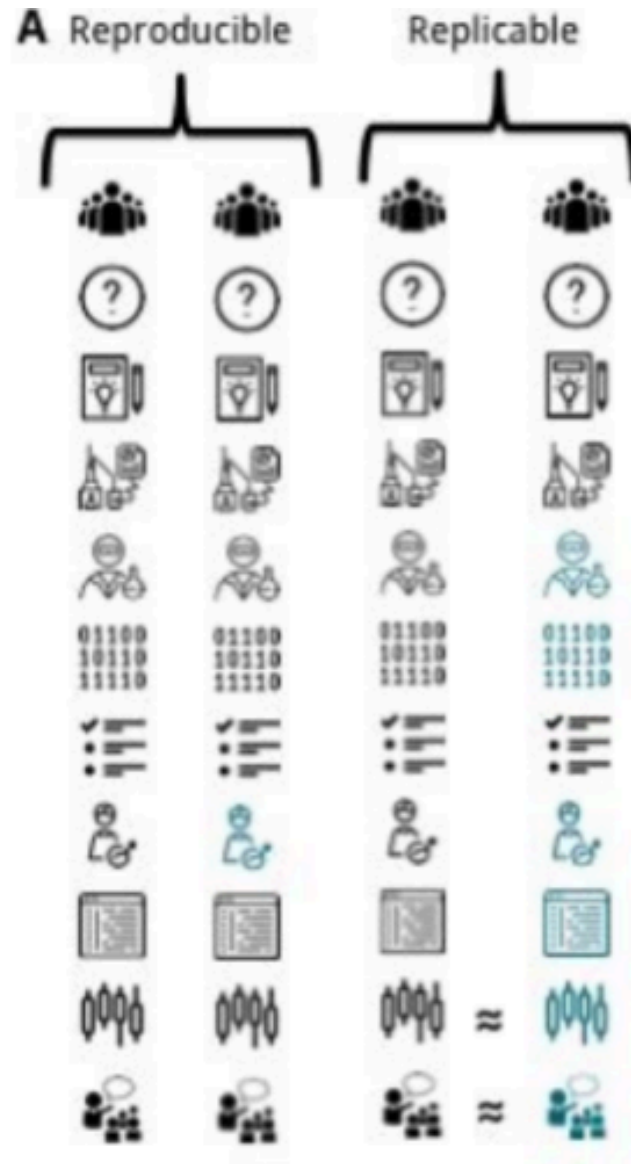
Code



Estimate



Claim

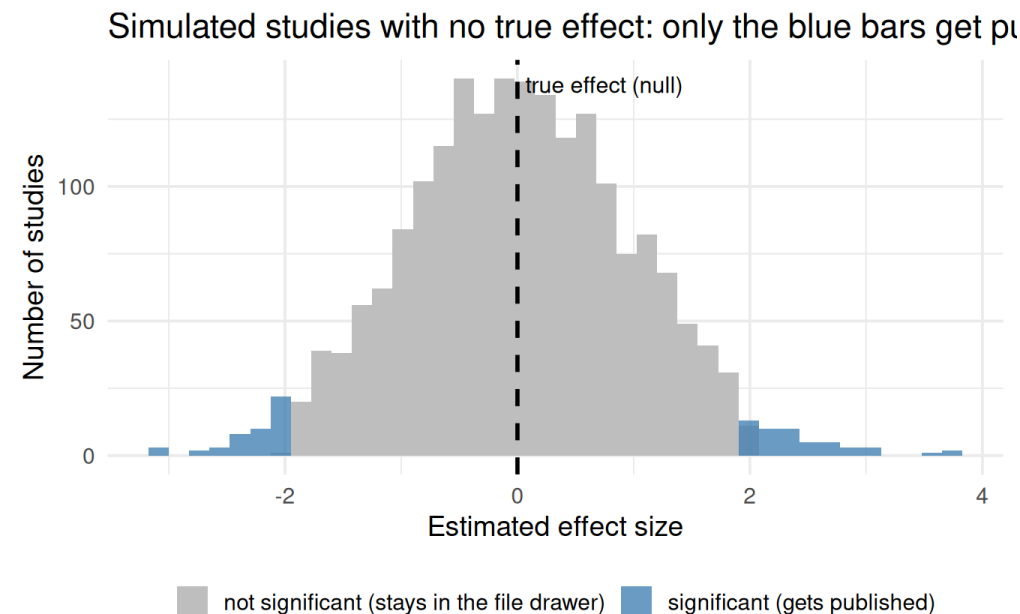


Why is Replication Hard?

- New source population with different characteristics
- Analytic plan not clearly defined
- Different interpretations of success or clinical utility
- Bad luck in sampling / file-drawer effect on first study / winner's curse

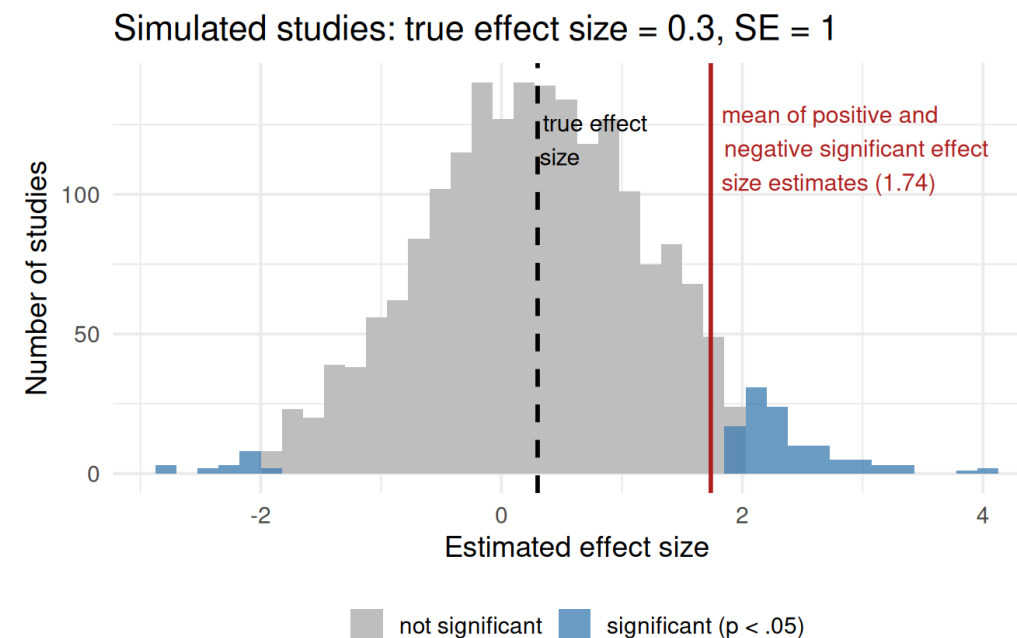
File Drawer Effect

- Studies with null or non-significant results are less likely to be submitted or accepted for publication
- The literature you see is a *selected* sample of all studies that were actually run
- Meta-analyses that rely on published studies can overestimate an effect because a hidden multiple hypothesis testing situation: null results never reported
- Contrast with winner's curse (next slide): file drawer effect biases *which* results you see, not the size of the effect within a given result



Winner's Curse (in the context of statistics)

- Among studies (or associations) that clear a significance threshold, the *estimated* effect size tends to be inflated relative to the *true* effect
- Occurs because, with noisy estimates, the ones that happen to exceed the threshold are disproportionately those where chance pushed the estimate up
- Distinct from the file drawer effect: winner's curse biases the *magnitude* of reported effects, not *whether* a result is reported at all



Winner's Curse (in the context of an auction)

Roughly:

- If you win as a buyer in an auction, you probably overestimated the value of what you bought
- (assuming that items at auction have the same actual value to all bidders)

Replication for Data Science Problems

Consider the source population

- Should a prediction model developed at Duke Hospital be transferable at UNC? Durham Regional Hospital? Columbia University?
- Should prediction model of risk of hospitalization developed before the Affordable Care Act be useful today?
- Should a prediction model built on data from Caucasians be useful for African Americans?
- How should a prediction model perform after new treatments are introduced?

Framingham Risk Score

- Predictors: age, sex, total cholesterol, HDL cholesterol, systolic blood pressure (and whether treated), smoking status
- Each predictor is converted to points using sex-specific tables
- Points are summed and mapped to an estimated 10-year probability of a cardiovascular event
- Risk categories: <5% low, 5-20% intermediate, >20% high

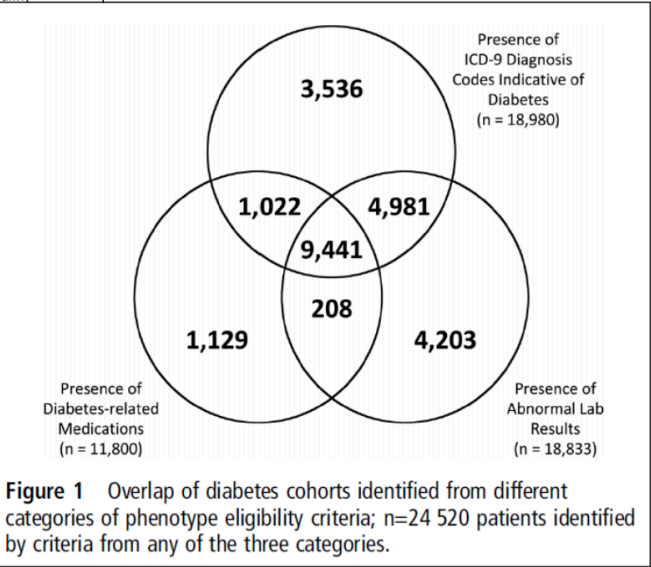
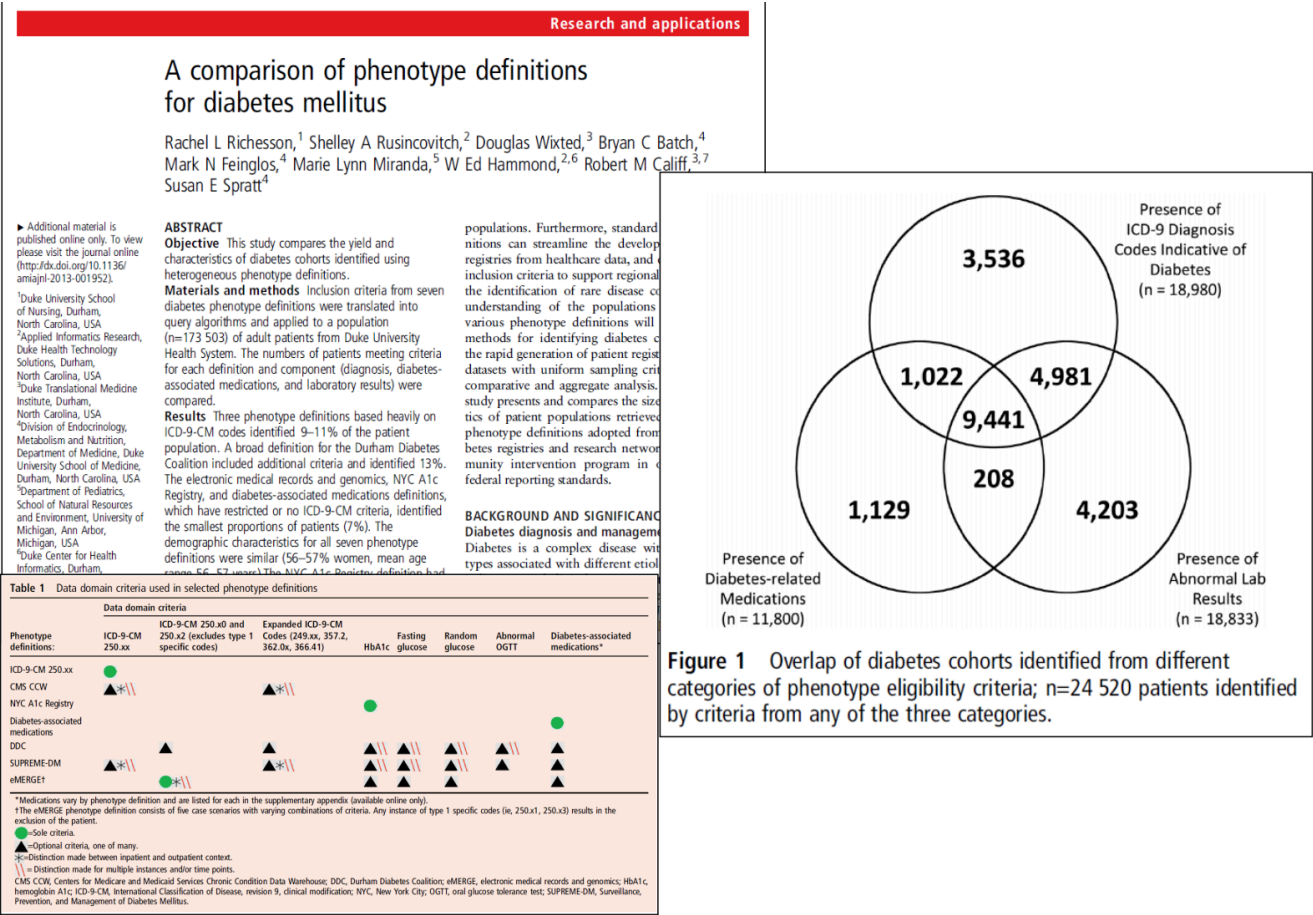
Framingham Risk Score

- Developed based on a cohort from Framingham MA started in 1948; predominantly white, middle-class cohort
- Focused on identifying *biological* risk factors (only 8 variables) (vs. possible societal determinants)
- Poorer performance in Asian and diabetic populations
- So “not replicable in a wider population”
- Usually if you specify “replicable” or “not replicable”, you should specify the population you have in mind and what other aspects are different

Replication

- Are the prediction and outcome variables defined and measured the same way?
- How are missing and outlying values handled in the new versus previous data set?
- Is the algorithm properly implemented?

Example of data definition issues: What is a Diabetic?



Richesson RL, Rusincovitch SA, Wixted D, Batch BC, Feinglos MN, Miranda ML, Hammond WE, Califf RM, Spratt SE. A Comparison of Phenotypic Definitions for Diabetes Mellitus. J Am Med Inf Assoc 2013 (epub ahead of print). <http://www.ncbi.nlm.nih.gov/pubmed/24026307>

Replication for Data Science Problems

Different Interpretations

- What is your definition of successful prediction model?
- How do you define clinical utility?

Reproducibility vs Replication

- Work should *always* be reproducible but may not always be replicable
- We can add elements to our work flow (coding, automatic reports, version control) to increase reproducibility
- If a study does not replicate that does not mean the previous study was wrong
- But it **may** mean the the previous study results are not generalizable and will not be useful in the wider real world

Part of reproducibility is making the data (and analysis) “FAIR”

Wilkinson MD, Dumontier M, Aalbersberg IJ, et al. (2016) “The FAIR Guiding Principles for scientific data management and stewardship,” *Scientific Data* 3:160018. <https://doi.org/10.1038/sdata.2016.18>

FAIR Principles: All research objects should be Findable, Accessible, Interoperable and Reusable (FAIR) both for machines and for people.